

The SQL fundamental (JOINS)

Relational Database

Primary key → is a unique identifier
→ different in each row
→ Not repeated

Primary key

Customers	
Customer_id	name
1	Rhops
2	Mbali

Foreign key → Can be repeated
→ Primary key in another table

Orders		
Order_id	Customer_id	amount
101	1	300
102	1	150
103	3	500

→ Can be repeated

16/03/2020

Time → line graph

Best selling products
Worst selling products

Task 1: Planning and architecture (Bright-coffee-shop-sales dataset)
Retail dataset

Miro
↓

How to plan using Miro?

Upload the plan on Github

What is ETL pipeline? (ETL = Extract, Transform, Load)

→ It is the mechanism by which ETL processes occur. Data pipelines are a set of tools and activities for moving data from one system, with its methods of data storage and processing to another system in which it can be stored and managed differently. (Databricks explanation)

The two main reasons to use ETL are for performance and visibility.

ETL pipeline → is an automated system that pulls raw data from multiple sources, cleans and converts it into a structured format, and loads it into a destination like a data warehouse. (AI explanation (google))

E
T
L
↓
x
t
r
a
c
t
↓
t
r
a
n
s
f
o
r
m
↓
l
o
a
d

ETL is best for smaller datasets requiring complex transformations before reaching the target.
Conversely, ELT (Extract, Load, Transform) is preferred for large, unstructured datasets where transformation occurs after loading into a scalable cloud data warehouse.

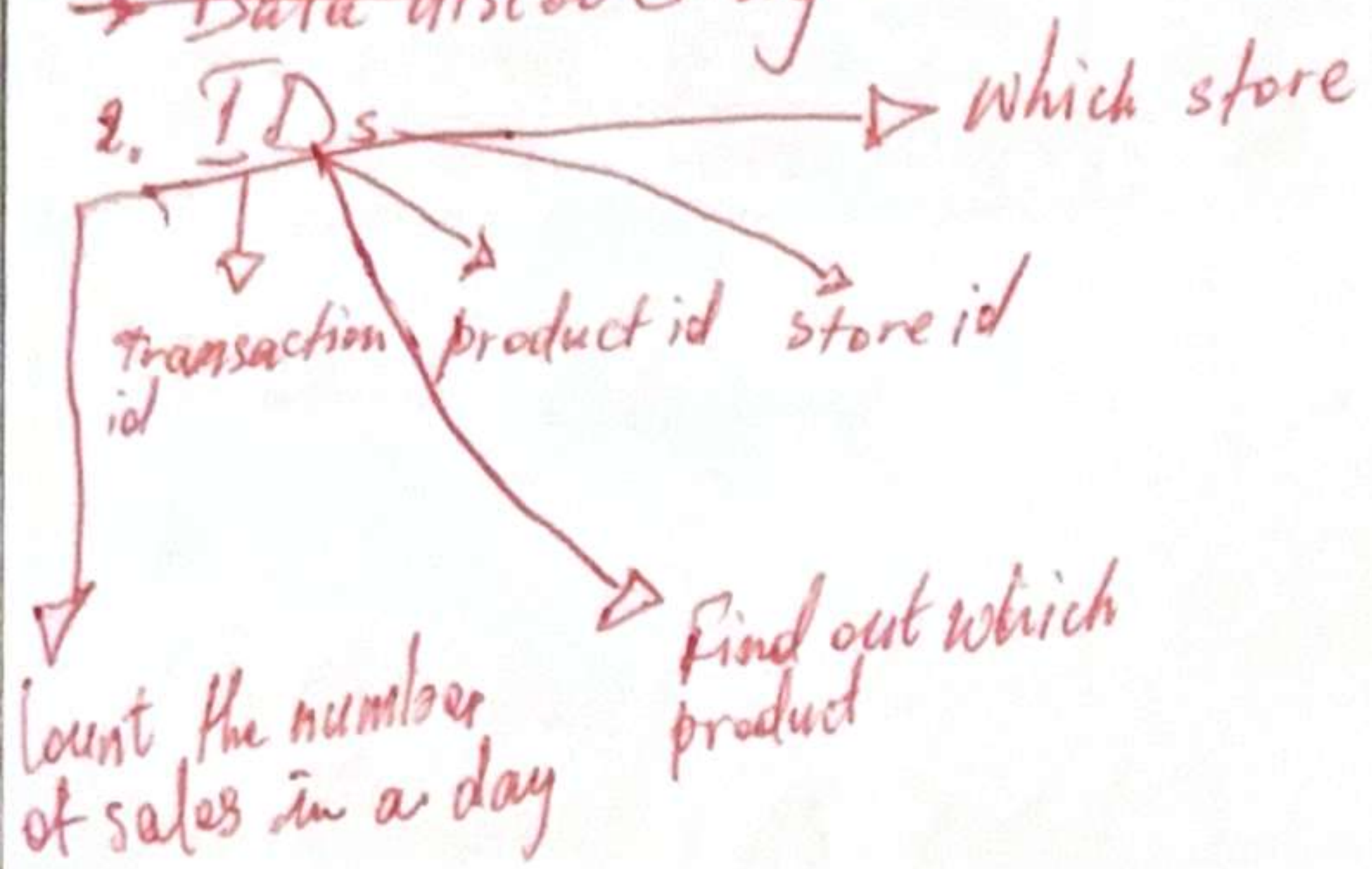
→ Loads the refined data into a destination, such as a data warehouse or data lakehouse (e.g. Snowflake, Redshift, BigQuery, Databricks)

→ Cleans, filters, joins, and reformats the data to match the target schema

→ Pulls raw data from various sources (SQL, SaaS, APIs, CRM)

→ Data discovering

2. IDs



3. Categorical columns

store location product category product type product details

Sales by store location

Sales

id	product	Category.	amount.
101	pen	Stationery	10
102	book	Stationery	15
103	Mouse	Electronics	100
104	Laptop	Electronics	200
105	Kettle	Appliances	90

Write an SQL script to calculate the money generated per category.

SELECT Category

FROM Sales

Sum(amount) AS total_amount

GROUP BY Category;

SELECT Category,

Sum(amount) AS total_amount

FROM Sales

GROUP BY category;

Category	total amount
Stationery	25
Electronics	300
Appliances	90

Having

SELECT AGG_FUN(Column1) AS new_column,
categorical_column

FROM table

GROUP BY categorical_column

HAVING AGG_FUN(Column1) condition;

Which category generates more than R100.

SELECT Category,

Sum(amount) AS total_amount

FROM Sales

GROUP BY category

HAVING total_amount > 100;

Aggregate functions

1. Sum()
2. Count()
3. Avg()
4. Min()
5. Max()

They perform a calculation on the rows of a specific column and return a single value.

GROUP BY



It groups the data into a specific categorical column.

HAVING



- Comes after the GROUP BY clause
- It is used to filter aggregated data
- Filters grouped data.

Table: Employees

ID	Name	Department	Salary
1	Rofhiwa	IT	100
2	Mbali	HR	20
3		HR	10
4		Sales	15
5		Sales	10

Write an SQL script to calculate the total salary paid to each department

```
SELECT Department
FROM Employees
Sum WHERE(Salary) AS total_salary
WHERE
GROUP BY Department;
```

```
SELECT Department
```

Department	total_salary
IT	100
HR	30
Sales	25

SQL CASE statement

25/02/2026

SELECT

CASE: WHEN condition 1 THEN result 1
 WHEN condition 2 THEN result 2

ELSE default_result

END AS new_column_name

FROM table;

→ You will get the default_result when all the outlined conditions are not satisfied.

Sales

Sale_id	Store	Price Amount
1	Pep	100
2	Pep	50
3	Mr price	80
4	Mr price	20

0-40 Low price

41-60 Medium price

>60 High price

SELECT Sale_id, store, price

CASE:

WHEN price BETWEEN 0 AND 40 THEN 'Low Price'

WHEN price BETWEEN 41 AND 60 THEN 'Medium price'

ELSE 'High price'

END AS price_group

FROM Sales;

Sale_id	Store	Price	price_group
1	Pep	100	High price
2	Pep	50	Medium price
3	Mr price	80	High price
4	Mr price	20	Low price

A case statement help us to classify, to group, to bucket our data
A case statement help us to create a categorical column from a numeric column based on specific conditions

```
SELECT product_name, price
```

```
  CASE:
```

```
    WHEN price BETWEEN 100 AND 1000 THEN 'Mid-range'
```

```
    WHEN price < 100 THEN 'Budget'
```

```
  ELSE 'Expensive'
```

```
END AS 'price_category'
```

```
FROM products;
```

SQL Logical operators

- AND
- OR
- NOT
- BETWEEN

We use it in the WHERE statement in order to filter the rows.

Day 7

SQL Operators → They operate in the WHERE clause

- We can use the operators within our WHERE statement in order to build the logic of filtering our data.
- Filtering is about excluding or including specific rows based on specific conditions

Logical operators

AND operator → It's an operator that we use to join together multiple conditions.
→ Both the conditions must be met.

SELECT column1,
column 2,
column 3

FROM database.schema.table

WHERE condition 1 AND condition 2;

Question

We want the name and marks of a student doing maths and got above 50%.

Curro.students.marks

Student Id	Name	Subject	Marks
101	Rofhi	Maths	99
102	Naledi	Maths	22
103	Mbali	Geo	50
104	Ally	Acc	40

SELECT name,
marks

FROM curro.students.ma

WHERE subject = 'Maths' AND marks > 50;

Name	Marks
Rofhi	99